



[14주차] 논문리뷰

EfficientNet

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

2. Related Work

3. Compound Model Scaling (제안 방법론)

1. Problem Formulation

2. Scaling Dimensions

3. Compound Scaling

4. EfficientNet Architecture

4. 실험 및 결과

1. Scaling Up MobileNets and ResNet

2. ImageNet Results for EfficientNet

3. Transfer Learning Results for EfficientNet

5. Discussion

6. 결론 (배운점)

7. Reference

KTO

1. Introduction

논문이 다루는 분야

해당 task에서 기존 연구 한계점

논문의 contributions

EfficientNet

1. Introduction



논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가

논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- **CV - Image Classification**

- 이미지 분류 성능을 향상시키기 위한 CNN 구조 설계
- compound coefficient를 제안
→ 모델의 깊이, 너비, 입력 이미지 해상도의 균형을 맞출 수 있는 새로운 스케일링 방법 제안

해당 task에서 기존 연구 한계점

- CNN에서 정확도를 높이기 위해 사용한 기존 방법
: Fixed Resource Budget : 모델 크기를 키움, 대표적으로 아래 3가지 방법
 - Depth : 모델 layer 수를 늘림
 - Width : filter (혹은 channel 수) 수를 늘림
 - Resolution : input image의 해상도(크기)를 높임
- 기존의 방법은 위 세가지 중 한가지 방법만을 선택해서, 정확도를 높여왔음

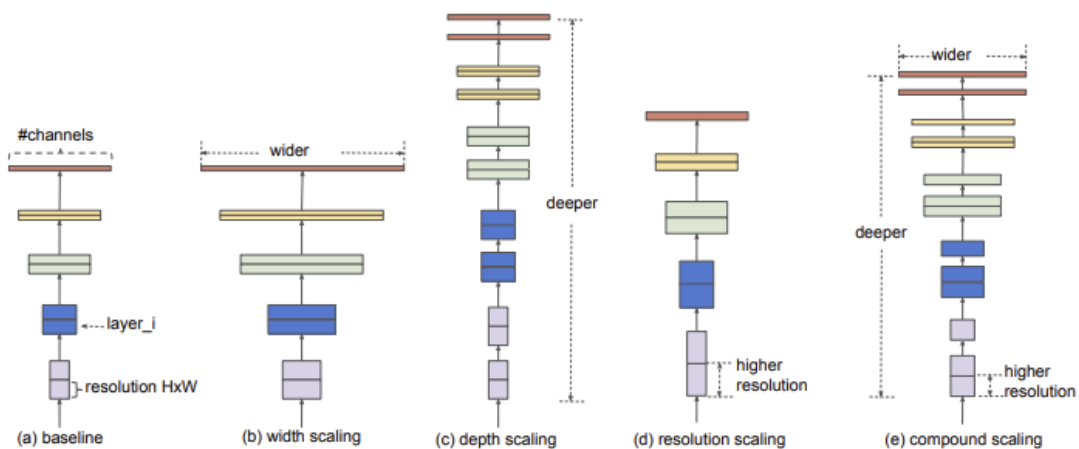


Figure 2. **Model Scaling.** (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

- Depth: ResNet-18부터 ResNet-200까지 layer 수를 증가시키며 정확도 향상
- Width: Wide Residual Network : ResNet의 conv filter 수 증가
- Image Resolution: GPipe : baseline model을 4배만큼 키움으로써 정확도 향상

⇒ 그러나 기존 Scaling 방법에는 가이드라인이 없으며, 어떤 종류를 얼마만큼 높여야 성능이 향상된다는 객관적인 지표가 없음

논문의 contributions

[본 연구에서 CNN의 정확도를 높이기 위해 사용한 방법]

1. Compound Scaling Method

- 'Process of Scaling Up CNN'
- "CNN의 더 높은 정확도와 효율성을 달성할 수 있는 Scaling 방법의 원칙이 있을까?"
- 네트워크의 width/depth/resolution의 balance를 맞추는 것이 중요하다는 것을 보여줌
- width/depth/resolution을 각각 일정한 비율로 증가하기만 하면 됨
- 실험 결과를 바탕으로, 간단하고 효율적인 compound scaling method를 제안
- 기존 연구는 요인(width/depth/resolution)을 임의의 값으로 증가시켰지만, 본 연구에서는 고정된 스케일링 계수 세트를 사용하여 각 요인을 균등하게 스케일링
- 컴퓨팅 리소스를 2^N 배 만큼 더 사용하고 싶으면, 파라미터들을 N승으로 증가

2. New Baseline - EfficientNet

- 기존의 MobileNets과 ResNet에 본 연구의 scaling method가 잘 작동됨
 - model scaling의 효과는 baseline network에 크게 의존
 - NAS(Neural Architecture Search)를 통해 새로운 baseline을 개발했음
⇒ EfficientNet
-

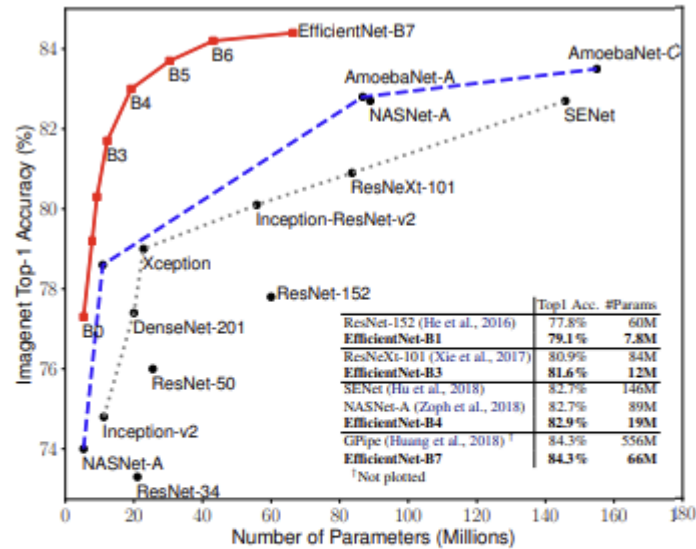


Figure 1. Model Size vs. ImageNet Accuracy. All numbers are for single-crop, single-model. Our EfficientNets significantly outperform other ConvNets. In particular, EfficientNet-B7 achieves new state-of-the-art 84.3% top-1 accuracy but being 8.4x smaller and 6.1x faster than GPipe. EfficientNet-B1 is 7.6x smaller and 5.7x faster than ResNet-152. Details are in Table 2 and 4.

- EfficientNet의 성능 비교 표 (Model Size VS ImageNet)
- EfficientNet-B7은 ImageNet 데이터에서 가장 성능이 좋은 정확도 84.3%을 달성, 기존의 CNN 보다 파라미터 수가 훨씬 적음

2. Related Work



Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

- ConvNet Accuracy
 - AlexNet 이후 더 깊고 크고 정확도 높은 모델 많이 발표됨. 이미지넷뿐 아니라 다른 데이터셋에서도 잘 작동. 정확도 높아짐과 같이 사용하는 자원도 늘어남
- ConvNet Efficiency
 - 깊은 ConvNet은 over-parameterized되는 경향이 있음.
 - 표율을 높이기 위한 여러 모델 압축 기법이 제안됐음
 - SqueezeNets, MobileNets, ShuffleNets 등

- Model Scaling
 - ResNet → 깊이 조정
 - MobileNet → 너비 조정
 - 또한 이미지 해상도 높아지면 정확도 높아짐(정보, 계산량이 모두 많아짐)
- ⇒ 효율적인 조합 아직 정립되지 않은 상태
- 본 논문에서는 새로운 스케일링 방법을 제안하여 너비, 깊이, 해상도 측면의 스케일링에서 균형을 맞출 수 있도록 하였다.

3. Compound Model Scaling (제안 방법론)



Introduction에서 언급된 내용과 동일하게 작성되어 있는가
 Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
 제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

1. Problem Formulation

- Conv Layer의 수식적 표현

$\mathcal{F}_i$: 연산자

Y_i : output tensor

X_i : input tensor (shape는 $\langle H_i, W_i, C_i \rangle$)

- H_i, W_i : spatial dimension

- C_i : channel dimension

$$\mathcal{N} = \mathcal{F}_k \odot \dots \odot \mathcal{F}_2 \odot \mathcal{F}_1(X_1) = \bigodot_{j=1 \dots k} \mathcal{F}_j(X_1)$$

- ConvNet의 여러개의 stage로 분할 → 각 stage의 layer들은 같은 architecture를 가짐 → 전체 ConvNet은 아래와 같이 정의할 수 있다.

$$\mathcal{N} = \bigodot_{i=1 \dots s} \mathcal{F}_i^{L_i}(X_{\langle H_i, W_i, C_i \rangle}) \quad (1)$$

$\mathcal{F}_i^{L_i}$: $\mathcal{F}_i$ 가 i 단계에서 L_i 번 반복되는 것

$\langle H_i, W_i, C_i \rangle$: i 층의 input tensor X 의 형태

- 일반적인 ConvNet은 best layer architecture인 Fi를 찾고자 하지만, model scaling은 기본 network에서 미리 정의된 Fi를 변경하지 않고 network의 Length(Li), width(Ci), resolution(Hi,Wi)을 확장하려 한다. 이 때 모든 layer는 일정한 비율로 균일하게 scaling되어야한다. 결국 본 연구자들의 목표는 최적화문제로 공식화될 수 있는 주어진 제약에서 모델 정확도를 극대화하는 것이다.

$$\begin{aligned}
 & \max_{d,w,r} \text{Accuracy}(\mathcal{N}(d, w, r)) \\
 & s.t. \quad \mathcal{N}(d, w, r) = \bigodot_{i=1 \dots s} \hat{\mathcal{F}}_i^{d \cdot \hat{L}_i}(X_{\langle r \cdot \hat{H}_i, r \cdot W_i, w \cdot \hat{C}_i \rangle}) \\
 & \quad \text{Memory}(\mathcal{N}) \leq \text{target_memory} \\
 & \quad \text{FLOPS}(\mathcal{N}) \leq \text{target_flops}
 \end{aligned} \tag{2}$$

2. Scaling Dimensions

- 최적화 문제의 어려움
 - d,w,r의 최적 값은 서로 의존하며, 서로 다른 리소스 제약 조건 하에서 값이 변화함
 - 예를 들어, GPU 성능이 좋으면 d,w,r은 매우 커도 되지만, 반대로 GPU 성능이 좋지 않으면 이 값은 작아져야 함.
 - 이러한 어려움으로 인해 기존 방법은 d,w,r 중 하나로 CNN을 확장

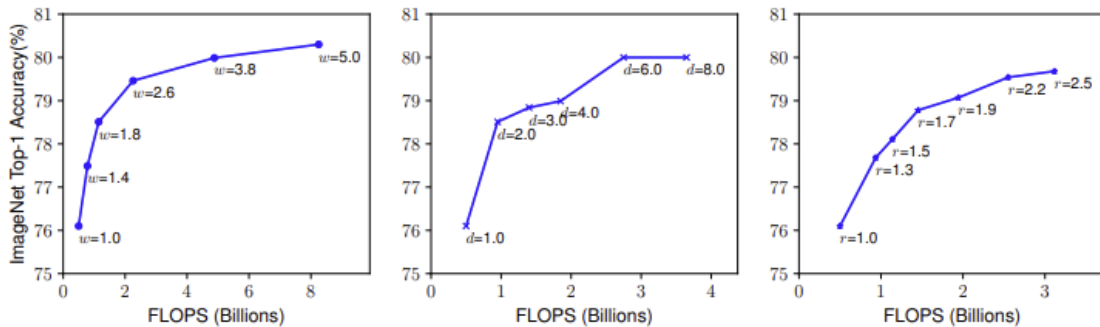


Figure 3. **Scaling Up a Baseline Model with Different Network Width (w), Depth (d), and Resolution (r) Coefficients.** Bigger networks with larger width, depth, or resolution tend to achieve higher accuracy, but the accuracy gain quickly saturate after reaching 80%, demonstrating the limitation of single dimension scaling. Baseline network is described in Table 1.

1. Depth 늘리기

- Deep한 CNN은 더 좋은 Feature map을 생성할 수 있음
 - 새로운 데이터에 대해 일반화 성능이 좋음
- network가 깊어질수록, Vanishing Gradient Problem 때문에 학습이 어려움
 - skip connections과 batch normalization을 적용한 ResNet이 등장
 - 여전히 기울기 소멸 문제가 존재

- 가운데 그래프 : 증가하다가 $d=6.0$ 부터 성능 향상 없음

2. Width 늘리기

- Wider Network는 세밀한 Feature를 잡을 수 있는 특징이 있으며, deep한 network에 비해 학습이 쉬움
- 깊이가 얇은 network는 higher level Features(더 좋은 Feature map)을 얻는 것이 어려움
- 왼쪽 그래프 : 증가하다가 $w=3.8$ 부터 성능 향상 폭이 작아짐

3. Resolution 늘리기

- 입력 이미지의 해상도(=height*Width)가 높을 수록, CNN은 이미지 속의 patterns을 더 잘 포착
- r 이 증가할 수록 성능이 향상

⇒ network의 width, depth 혹은 resolution의 차원을 scaling up하는 것은 정확도 향상에 도움이 되지만, 모델이 커질수록(=bigger FLOPS) 성능 향상 폭이 작아짐

3. Compound Scaling

- 모든 차원(w, d, r)을 scaling 해야 하는 이유
- different scaling dimensions은 독립적이지 않음
- 고해상도 이미지 입력
 - 더 큰 receptive fields는 큰 이미지에서 더 많은 픽셀을 포함하는 유사한 feature를 추출할 수 있도록, 네트워크 depth를 더 늘려야 함
 - 또한 고해상도 이미지에서 더 많은 픽셀의 패턴을 캡처할 수 있도록 네트워크의 width도 늘려야 함
- 이러한 직관들은 기존의 단일 차원 스케일링(w, d, r 중 하나만 스케일링)보다는 모든 차원(w, d, r 모두)에 다른 값을 스케일링 함으로써 균형을 맞추는 필요가 있음을 시사함
위를 검증하기 위해, 우리는 아래 그림과 같이 서로 다른 depths, resolutions에서 scaling한 결과를 비교함

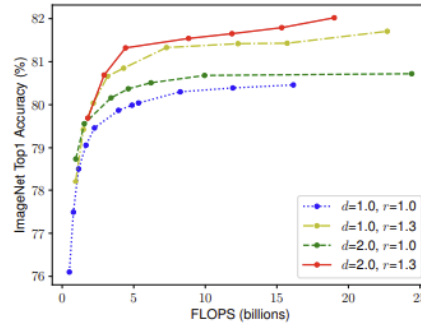


Figure 4. Scaling Network Width for Different Baseline Networks. Each dot in a line denotes a model with different width coefficient (w). All baseline networks are from Table 1. The first baseline network ($d=1.0, r=1.0$) has 18 convolutional layers with resolution 224×224 , while the last baseline ($d=2.0, r=1.3$) has 36 layers with resolution 299×299 .

- 첫번째 Baseline network(파란색, $d=1.0, r=1.0$)는 18개의 conv layer와 224×224 해상도
- 마지막 Baseline(빨간색, $d=2.0, r=1.3$)은 36개의 conv layer와 299×299 해상도
- 네트워크가 깊어지고, 고해상도가 될수록 정확도가 향상
- 더 나은 정확성 및 효율성을 위해 CNN의 depth, width 및 resolution의 모든 차원의 균형을 맞추는 것이 중요

$$\begin{aligned}
 \text{depth: } d &= \alpha^\phi \\
 \text{width: } w &= \beta^\phi \\
 \text{resolution: } r &= \gamma^\phi \\
 \text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 &\approx 2 \\
 \alpha \geq 1, \beta \geq 1, \gamma &\geq 1
 \end{aligned} \tag{3}$$

- α, β, γ : small grid search에 의해 결정되는 상수
- compound coefficient ϕ : 사용자의 리소스(GPU 성능)에 따라 사용자가 조절
- depth를 2배 $\rightarrow$ FLOPS가 2배 / **width와 resolution 2배 $\rightarrow$ 4배**
- total FLOPS $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \rightarrow$ 어떠한 ϕ 값을 설정해도, total FLOPS는 2^ϕ 만큼 증가

4. EfficientNet Architecture

- Baseline Network의 중요성

Table 1. EfficientNet-B0 baseline network – Each row describes a stage i with $\hat{L}_i$ layers, with input resolution $(\hat{H}_i, \hat{W}_i)$ and output channels $\hat{C}_i$. Notations are adopted from equation 2.

Stage i	Operator $\hat{\mathcal{F}}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

- Model Scaling은 layer Operators $\mathcal{F}_i$ 가 고정 → 좋은 baseline network를 가지는 것은 매우 중요
- 새로 고안한 Compound scaling method의 효과를 극명히 드러내기 위해 새로운 mobile-size baseline를 설계 = EfficientNet
 - NAS에서 정확도 및 FLOPS를 모두 챙길 수 있도록 최적화
 - 가장 base한 네트워크 = EfficientNet-B0
 - scaling method
 - STEP 1: we first fix $\phi = 1$, assuming twice more resources available, and do a small grid search of α, β, γ based on Equation 2 and 3. In particular, we find the best values for EfficientNet-B0 are $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$, under constraint of $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$.
 - STEP 2: we then fix α, β, γ as constants and scale up baseline network with different ϕ using Equation 3, to obtain EfficientNet-B1 to B7 (Details in Table 2).
- ϕ 를 1로 고정 → 최적의 α, β, γ 구함 → 이 값들을 고정 → 서로 다른 ϕ 로 스케일링 → EfficientNet B1에서 B7까지 만듦

4. 실험 및 결과



Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

- EfficientNet B0 ~ B7의 ImageNet data 성능
→ 비슷한 성능을 가지는 모델 대비 parameter의 수가 적고 FLOPs가 적다)

Table 2. **EfficientNet Performance Results on ImageNet** (Russakovsky et al., 2015). All EfficientNet models are scaled from our baseline EfficientNet-B0 using different compound coefficient ϕ in Equation 3. ConvNets with similar top-1/top-5 accuracy are grouped together for efficiency comparison. Our scaled EfficientNet models consistently reduce parameters and FLOPS by an order of magnitude (up to 8.4x parameter reduction and up to 16x FLOPS reduction) than existing ConvNets.

Model	Top-1 Acc.	Top-5 Acc.	#Params	Ratio-to-EfficientNet	#FLOPs	Ratio-to-EfficientNet
EfficientNet-B0	77.1%	93.3%	5.3M	1x	0.39B	1x
ResNet-50 (He et al., 2016)	76.0%	93.0%	26M	4.9x	4.1B	11x
DenseNet-169 (Huang et al., 2017)	76.2%	93.2%	14M	2.6x	3.5B	8.9x
EfficientNet-B1	79.1%	94.4%	7.8M	1x	0.70B	1x
ResNet-152 (He et al., 2016)	77.8%	93.8%	60M	7.6x	11B	16x
DenseNet-264 (Huang et al., 2017)	77.9%	93.9%	34M	4.3x	6.0B	8.6x
Inception-v3 (Szegedy et al., 2016)	78.8%	94.4%	24M	3.0x	5.7B	8.1x
Xception (Chollet, 2017)	79.0%	94.5%	23M	3.0x	8.4B	12x
EfficientNet-B2	80.1%	94.9%	9.2M	1x	1.0B	1x
Inception-v4 (Szegedy et al., 2017)	80.0%	95.0%	48M	5.2x	13B	13x
Inception-resnet-v2 (Szegedy et al., 2017)	80.1%	95.1%	56M	6.1x	13B	13x
EfficientNet-B3	81.6%	95.7%	12M	1x	1.8B	1x
ResNeXt-101 (Xie et al., 2017)	80.9%	95.6%	84M	7.0x	32B	18x
PolyNet (Zhang et al., 2017)	81.3%	95.8%	92M	7.7x	35B	19x
EfficientNet-B4	82.9%	96.4%	19M	1x	4.2B	1x
SENet (Hu et al., 2018)	82.7%	96.2%	146M	7.7x	42B	10x
NASNet-A (Zoph et al., 2018)	82.7%	96.2%	89M	4.7x	24B	5.7x
AmoebaNet-A (Real et al., 2019)	82.8%	96.1%	87M	4.6x	23B	5.5x
PNASNet (Liu et al., 2018)	82.9%	96.2%	86M	4.5x	23B	6.0x
EfficientNet-B5	83.6%	96.7%	30M	1x	9.9B	1x
AmoebaNet-C (Cubuk et al., 2019)	83.5%	96.5%	155M	5.2x	41B	4.1x
EfficientNet-B6	84.0%	96.8%	43M	1x	19B	1x
EfficientNet-B7	84.3%	97.0%	66M	1x	37B	1x
GPipe (Huang et al., 2018)	84.3%	97.0%	557M	8.4x	-	-

We omit ensemble and multi-crop models (Hu et al., 2018), or models pretrained on 3.5B Instagram images (Mahajan et al., 2018).

- 파라미터 수(FLOPs 크기) 별로 모델들을 정렬
- **EfficientNet-B7**은 기존의 state-of-the-art인 GPipe와 동일한 성능
 - BUT, 파라미터 수가 8.4배 가량 적음

1. Scaling Up MobileNets and ResNet

- MobileNet과 ResNet에 적용시켜 보았을 때 단일 dimension scaling보다 compound scaling의 효과가 좋았다.

Table 3. Scaling Up MobileNets and ResNet.

Model	FLOPS	Top-1 Acc.
Baseline MobileNetV1 (Howard et al., 2017)	0.6B	70.6%
Scale MobileNetV1 by width ($w=2$)	2.2B	74.2%
Scale MobileNetV1 by resolution ($r=2$)	2.2B	72.7%
compound scale ($d=1.4, w=1.2, r=1.3$)	2.3B	75.6%
Baseline MobileNetV2 (Sandler et al., 2018)	0.3B	72.0%
Scale MobileNetV2 by depth ($d=4$)	1.2B	76.8%
Scale MobileNetV2 by width ($w=2$)	1.1B	76.4%
Scale MobileNetV2 by resolution ($r=2$)	1.2B	74.8%
MobileNetV2 compound scale	1.3B	77.4%
Baseline ResNet-50 (He et al., 2016)	4.1B	76.0%
Scale ResNet-50 by depth ($d=4$)	16.2B	78.1%
Scale ResNet-50 by width ($w=2$)	14.7B	77.7%
Scale ResNet-50 by resolution ($r=2$)	16.4B	77.5%
ResNet-50 compound scale	16.7B	78.8%

2. ImageNet Results for EfficientNet

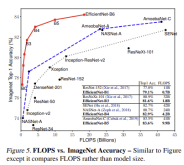


Table 4. Inference Latency Comparison – Latency is measured with batch size 1 on a single core of Intel Xeon CPU E5-2690.

Acc. @ Latency		Acc. @ Latency	
ResNet-152	77.8% @ 0.554s	GPipe	84.3% @ 19.0s
EfficientNet-B1	78.8% @ 0.098s	EfficientNet-B7	84.4% @ 3.1s
Speedup	5.7x	Speedup	6.1x

- 위에서 봤던 ImageNet에 대한 모델 별 정확도 비교 그래프
- EfficientNet은 정확도와 연산량을 모두 잡음 → 효율적인 network

3. Transfer Learning Results for EfficientNet

- 전이 학습 : ImageNet과 같은 대량의 데이터에서 pre-train을 하고, 새로운 데이터 (CIFAR10, Birdsnap 등등)에서 fine-tuning하는 학습 방법
- 전이 학습의 결과도 EfficientNet의 성능이 우위에 있음
- Transfer Learning에 사용한 데이터셋 개요

Table 6. Transfer Learning Datasets.

Dataset	Train Size	Test Size	#Classes
CIFAR-10 (Krizhevsky & Hinton, 2009)	50,000	10,000	10
CIFAR-100 (Krizhevsky & Hinton, 2009)	50,000	10,000	100
Birdsnap (Berg et al., 2014)	47,386	2,443	500
Stanford Cars (Krause et al., 2013)	8,144	8,041	196
Flowers (Nilsback & Zisserman, 2008)	2,040	6,149	102
FGVC Aircraft (Maji et al., 2013)	6,667	3,333	100
Oxford-IIIT Pets (Parkhi et al., 2012)	3,680	3,369	37
Food-101 (Bossard et al., 2014)	75,750	25,250	101

- Transfer Learning 결과 (모델 별 정확도 비교)

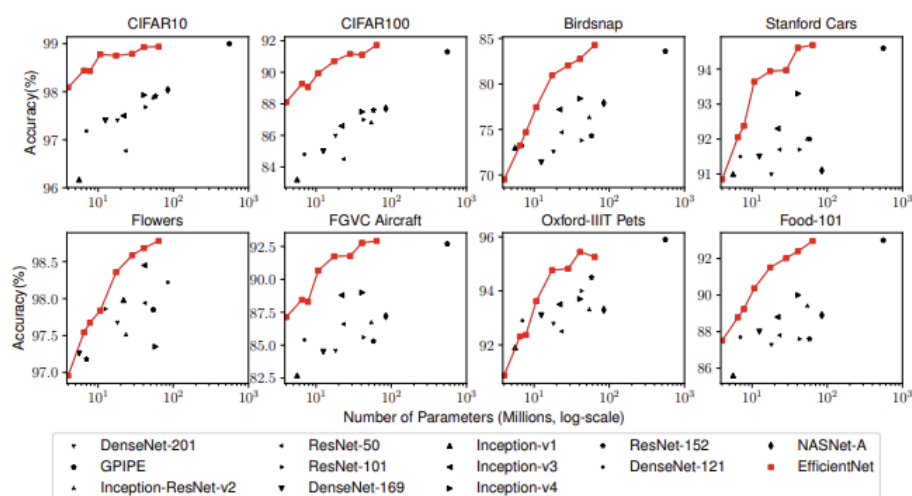


Figure 6. Model Parameters vs. Transfer Learning Accuracy – All models are pretrained on ImageNet and finetuned on new datasets.

Table 5. EfficientNet Performance Results on Transfer Learning Datasets. Our scaled EfficientNet models achieve new state-of-the-art accuracy for 5 out of 8 datasets, with 9.6x fewer parameters on average.

	Model	Comparison to best public-available results				Comparison to best reported results			
		Acc.	#Param	Our Model	Acc.	#Param(ratio)	Model	Acc.	#Param
CIFAR-10	NASNet-A	98.0%	85M	EfficientNet-B0	98.1%	4M (21x)	[†] Gpipe	99.0%	556M
CIFAR-100	NASNet-A	87.5%	85M	EfficientNet-B0	88.1%	4M (21x)	Gpipe	91.3%	556M
Birdsnap	Inception-v4	81.8%	41M	EfficientNet-B5	82.0%	28M (1.5x)	Gpipe	83.6%	556M
Stanford Cars	Inception-v4	93.4%	41M	EfficientNet-B3	93.6%	10M (4.1x)	[‡] DAT	94.8%	-
Flowers	Inception-v4	98.5%	41M	EfficientNet-B5	98.5%	28M (1.5x)	DAT	97.7%	-
FGVC Aircraft	Inception-v4	90.9%	41M	EfficientNet-B3	90.7%	10M (4.1x)	DAT	92.9%	-
Oxford-IIIT Pets	ResNet-152	94.5%	58M	EfficientNet-B4	94.8%	17M (5.6x)	Gpipe	95.9%	556M
Food-101	Inception-v4	90.8%	41M	EfficientNet-B4	91.5%	17M (2.4x)	Gpipe	93.0%	556M
Geo-Mean						(4.7x)			
									(9.6x)

[†]Gpipe (Huang et al., 2018) trains giant models with specialized pipeline parallelism library.

[‡]DAT denotes domain adaptive transfer learning (Ngiam et al., 2018). Here we only compare ImageNet-based transfer learning results.

Transfer accuracy and #params for NASNet (Zoph et al., 2018), Inception-v4 (Szegedy et al., 2017), ResNet-152 (He et al., 2016) are from (Kornblith et al., 2019).

- 더 적은 parameter로 5개의 dataset에서 accuracy SOTA를 기록했으며 전이학습에서도 좋은 성능을 보인다.

5. Discussion

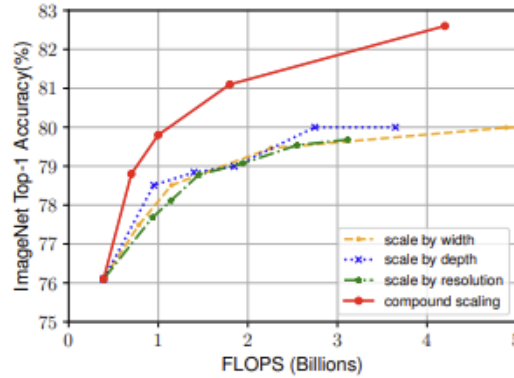


Figure 8. Scaling Up EfficientNet-B0 with Different Methods.

- 본문에서 제시한 compound scaling을 적용한 것이 성능이 제일 좋음

Table 7. Scaled Models Used in Figure 7.

Model	FLOPS	Top-1 Acc.
Baseline model (EfficientNet-B0)	0.4B	77.3%
Scale model by depth ($d=4$)	1.8B	79.0%
Scale model by width ($w=2$)	1.8B	78.9%
Scale model by resolution ($r=2$)	1.9B	79.1%
Compound Scale ($d=1.4, w=1.2, r=1.3$)	1.8B	81.1%

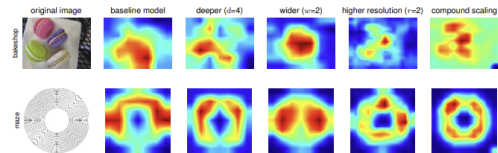


Figure 7. Class Activation Map (CAM) (Zhou et al., 2016) for Models with different scaling methods. Our compound scaling method allows the scaled model (last column) to focus on more relevant regions with more object details. Model details are in Table 7.

- CAM : 이미지 내에서 어떤 부분이 활성화되는지 시각화
- CAM 결과 역시 compound scaling을 적용한 것이 더 효과가 좋음

6. 결론 (배운점)



연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- 기존의 CNN scaling 방법 : depth, width, resolution 중 1,2개만을 고려
→ 정확도와 효율성을 모두 챙기지 못함 (ex : 네트워크 깊이가 깊어질수록 성능 향상 폭이 감소)
- 이러한 이슈를 해결하기 위해, 본 연구에서는 효과적인 scaling 방법인 compound scaling을 제안함
- compound scaling을 MobileNet, ResNet 그리고 새로 개발한 EfficientNet에 적용함으로써, SOTA 정확도를 달성, 파라미터 수와 FLOPs이 더 작아짐 (ImageNet, 전이 학습 모두)
- compound scaling을 통해 network를 보다 효율적으로 scaling 할 수 있게 됨

7. Reference

- 수식 참고 : <https://rahites.tistory.com/97>
- 전체적인 내용 이해
 - <https://ropiens.tistory.com/110>
 - <https://velog.io/@jus6886/%EB%85%BC%EB%AC%B8-%EB%A6%AC%EB%B7%B0-EfficientNet-Rethinking-Model-Scaling-for-Convolutional-Neural-Networks>

KTO

1. Introduction



논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- LLMs의 human alignment

해당 task에서 기존 연구 한계점

- LLM의 경우 RLHF, DPO 등의 정렬 기법들이 단순 지도학습 미세조정(SFT)만 수행하는 것보다 일관되게 나은 성과를 보임
 - 그러나 인간 피드백은 preference의 형태로만 주로 다뤄짐(x에 대해 y1보다 y2가 낫다)
 - 좋다/나쁘다 등의 신호데이터는 수집이 어렵고 비용이 많이 드는 데이터 형태
 - BUT, 현재 정렬 방식은 선호 데이터가 있어야 제대로 작동(RLHF, DPO)
 - 질문
 - 이 방법들이 왜 잘 작동하는가?
 - 피드백이 반드시 preference 형태여야 하는가?
- Prospect theory의 관점에서 정렬 문제 재해석

논문의 contributions

- 전망 이론
 - 기대값을 최대화하지 않는 불확실한 사건에 대해 어떻게 결정을 내리는지를 설명
 - 인간이 확률 변수들을 편향되지만 일정한 방식으로 인식함을 설명함
 - 어떤 기준점(reference point)을 기준으로 할 때, 인간은 이득보다 손실에 더 민감하게 반응한다는 손실 회피(loss aversion) 성향이 있음.

⇒ 본 연구에서는 DPO나 PPO-Clip과 같은 대표적인 정렬 방법들이 이러한 인간의 편향을 암묵적으로 모델링하고 있음을 보여줌 (이러한 기법들이 데이터에 상관없이 좋은 성능을 보이는 이유 중 하나)

⇒ 편향을 보다 일반적으로 반영하는 손실 함수 계열을 HALO(Human-Aware Losses)라고 정의

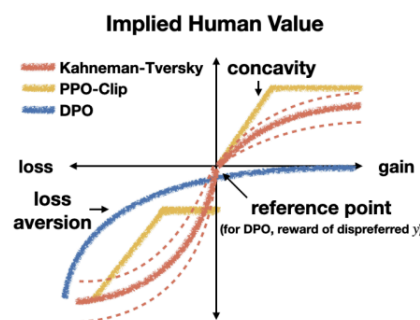


Figure 1. The utility that a human gets from the outcome of a random variable, as implied by different human-aware losses (HALOs). Notice that the implied value functions share properties such as loss aversion with the canonical human value function in prospect theory (Tversky & Kahneman, 1992).

- 인간이 무작위 결과(random variable)의 결과로부터 느끼는 가치(utility)를 여러 종류의 Human-Aware Loss (HALO)가 암묵적으로 어떻게 모델링하고 있는지 시각화한 그림
-
- HALO가 항상 기존 비-HALO보다 더 낫다고 단정 지을 수는 없음.
 - BUT, 우리는 기존 기법들 중에서도 HALO 조건을 만족하는 손실 함수가 그렇지 않은 것보다 일관되게 더 좋은 성능을 보임을 확인했음
 - 간단한 설정에서도 DPO가 상당히 높은 성능을 낼 수 있음을 발견
 - 더미 보상을 사용하는 오프라인 PPO 변형 → DPO와 비슷한 성능
 - 선호 데이터가 실제로 필요 없을 수도 있음을 시사
 - 손실 함수 자체의 직관적인 설계가 충분히 중요하다는 점을 드러냄

- DPO에는 미치지 못함.
- 30B 규모의 LLM에서는 하이퍼파라미터에 민감하게 반응하여 실제 사용이 어렵다는 단점
- 이론적인 접근을 바탕으로 HALO를 새롭게 정의
- 인간 효용 모델(human utility model)을 활용
 - 인간이 불확실한 금전적 결과에 대해 어떻게 의사결정을 하는지 설명
 - ⇒ Kahneman-Tversky Optimization (KTO) : 대부분의 기존 방식이 단순히 선호의 로그우도(likelihood)를 최대화하는 것과 달리, 생성 결과의 인간 효용을 직접 최대화하는 방식
 - KTO는 출력이 좋은지 나쁜지에 대한 이진 신호(binary signal)만 필요
 - 실제 환경에서 더 많은 데이터를, 더 저렴하고 빠르게 수집
 - 실제 프로덕션 환경에서 정렬을 적용하고 빠르게 반복 개선하는 데 유리

[주요발견]

- KTO는 DPO의 성능을 1B에서 30B scale에 걸쳐 대등하거나 능가
- KTO는 데이터 불균형에도 강건
- 사전 학습이 충분히 잘 된 모델에서는 SFT 없이 바로 KTO 적용이 가능

⇒ 단 하나의 HALO가 항상 최고는 X, 어떤 HALO가 적합한지는 태스크에 맞는 편향 구조(inductive bias)에 따라 달라짐. 하나의 방식만 고수하기보다는, 손실 함수를 상황에 따라 조정하는 것이 바람직함